

Canonical SOP

- 1. Identity rows with 1
- 2. Write out minterms
 $\bar{x}$ if 0, x if 1
ex. 001 $\rightarrow \bar{x}_2 \bar{x}_1 x_0$
- 3. OR them up
ex. $x_2 \bar{x}_1 + x_2 x_1$

Canonical POS

- 1. Identity rows with 0
- 2. Write out maxterms
 $\bar{x}$ if 1, x if 0
ex. 001 $\rightarrow x_2 + x_1 + \bar{x}_0$
- 3. AND them up
ex. $(x_2 + \bar{x}_1)(\bar{x}_2 + \bar{x}_1)$

DeMorgan's

POS, $(x_1 + x_2)(x_3 + x_4) = \overline{\overline{(x_1 + x_2)} + \overline{(x_3 + x_4)}}$

$x_1, x_2 \rightarrow \text{AND} \rightarrow \text{OR}$ (AND, AND)
 $x_3, x_4 \rightarrow \text{AND} \rightarrow \text{OR}$ (NOR, NOR)

SOP, $x_1 x_2 + x_3 x_4 = \overline{\overline{(x_1 x_2)} \cdot \overline{(x_3 x_4)}}$

$x_1, x_2 \rightarrow \text{AND} \rightarrow \text{OR}$ (AND, OR)
 $x_3, x_4 \rightarrow \text{AND} \rightarrow \text{OR}$ (NAND, NAND)

Karnaugh Maps

- Step 1: Find PIs (circle) $\rightarrow$ Implicants
 - Step 2: Identify essential PIs, Product terms covered by f include in cover $\rightarrow$ Prime implicants
 - Step 3: Choose suitable PIs, s.t. Implicant for which it is
a) all minterms covered
b) cost minimized
impossible to remove any literal and still be valid
- * notes: circle over/apply to find all POS
- Gray's code
00 01 11 10

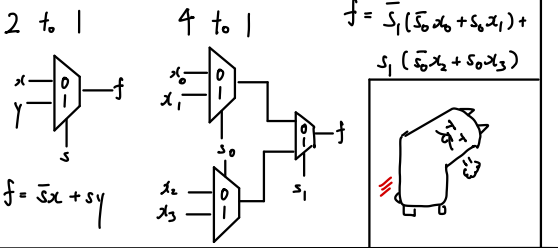
Definitions

Cover: Any set of implicants that includes all minterms of f

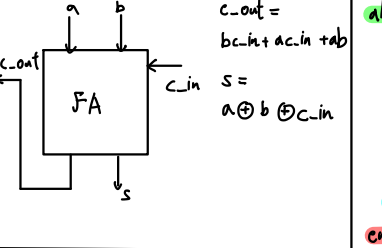
Consensus Theorem

$(x \cdot y) + (\bar{x} \cdot z) + (y \cdot z) = (x \cdot y) + (\bar{x} \cdot z)$
or $(x+y) \cdot (\bar{x}+z) \cdot (y+z) = (x+y) \cdot (\bar{x}+z)$

Mux



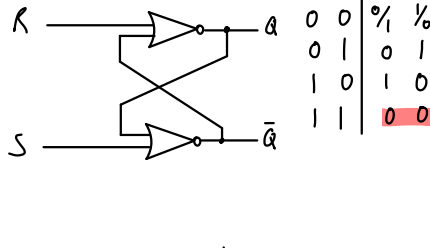
Full Adder



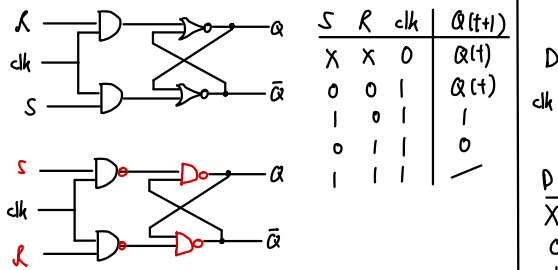
Case statement

always_comb
begin
case (logic)
2'b00: x = ...;
2'b01: x = ...;
:
default: x = 'b0;
endcase
end

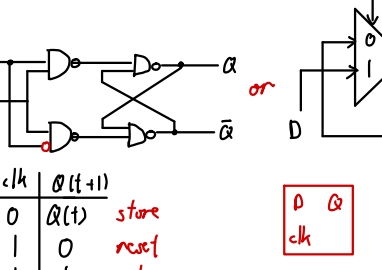
RS Latch



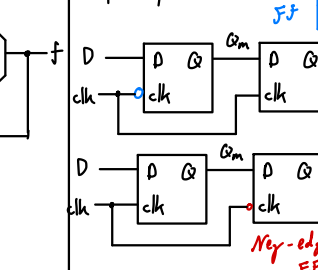
Gated RS Latch



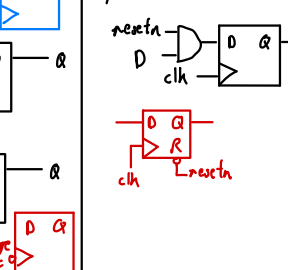
Gated D Latch



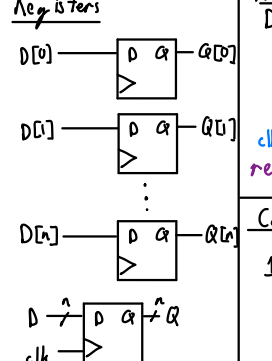
Flip Flops



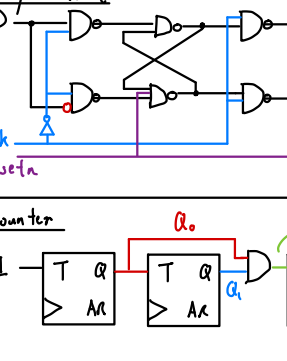
Sync Resets



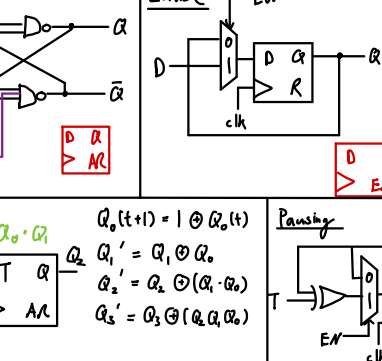
Registers



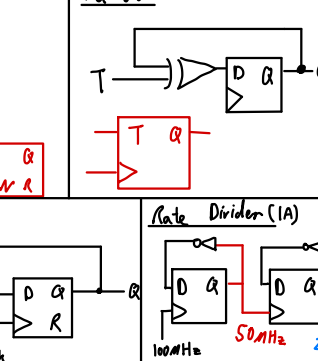
A-sync Reset



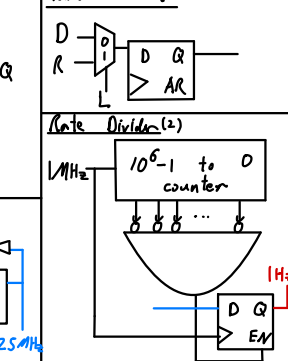
Enable



TQ FF



Parallel Load



Verilog D-latch

```
module D_latch(input logic D, clk, output logic Q);  
    always_latch  
    if (clk == 1)  
        Q = D;  
endmodule
```

Verilog FF

```
module D_FF(input logic clk, D, output logic Q);  
    always_ff @(posedge clk)  
        Q <= D;  
endmodule
```

Verilog Sync Reset

```
module Sync_Reset  
:  
    always_ff @(posedge clk)  
        if (!resetn) Q <= 1'b0;  
        else Q <= D;  
endmodule
```

Verilog A-Sync Reset

```
module A_Sync_Reset  
always_ff @(posedge clk, negedge resetn)  
begin  
    if (!resetn) Q <= 1'b0;  
    else Q <= D;  
end  
endmodule
```

Verilog Counter

```
module Counter  
always_ff @(posedge clk, negedge resetn)  
begin  
    if (!resetn) // reset  
        Q <= 4'b0;  
    else if (L) // parallel load  
        Q <= R;  
    else if (EN) // if enabled, ++  
        Q <= Q + 1;  
end  
end
```

Verilog FSM

```
module FSM(...)  
typedef enum logic [1:0] {A, B, C, D} state_type  
state_type ps, ns;  
  
1) JFS  
always_ff @(posedge clk, negedge resetn)  
if (!resetn) ps <= A;  
else ps <= ns;  
  
2) State Table  
case (ps)  
A: if (ns == B)  
    else ns = A;  
B: ...  
endcase  
  
3) Output  
assign z = (ps == D)  
endmodule
```

Verilog shift

```
RS: a >> b  
LS: a << b
```

do file format

```
vlib work  
vlog filename.v  
vsim modulename  
  
log -r { /* }  
add wave { /* }  
force {object-name} {value}  
run {time-steps} [{time-units}]  
force {SW[0]} 0  
run 10 ns
```

Periodic signals

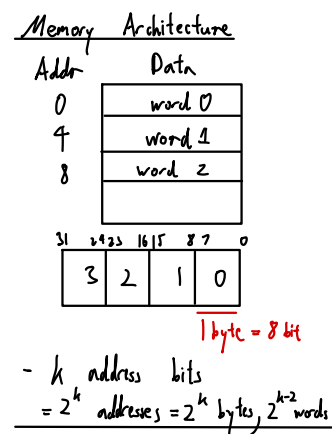
```
force {object-name} {value-1} {time-1}, {value-2} {time-2} -r {time-3}  
force {clk} 0 0ns, 1 {5ns} -r 10ns
```

Verilog Misc.

```
Reduction: &a = a[0] & a[1] & ...  
Concatenation: y = {a[2:1], b[3]} -> a[2] a[1] b[3]  
Parameter s: #(parameter WIDTH = 8)  
input logic [WIDTH-1:0] a;
```

Verilog processor

- Control Path (FSM)
- Data path
- 1) Choose item at mux
- 2) Carry out ALU math
- 3) Store item in reg.



Program Flow

IF, THEN, ELSE **FOR/WHILE**

IF: bgt s8, s9, THEN **LOOP:** bgt s8, s9, DONE

ELSE: # else code # CODE

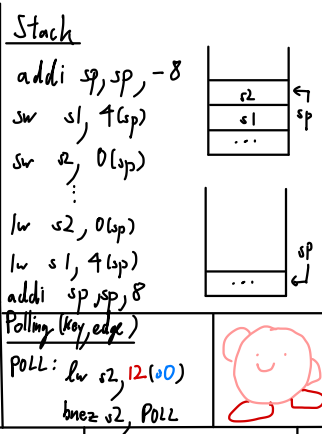
AFTER **j AFTER** **addi s8, s8, 1** **j LOOP**

THEN: # then code **addi s8, s8, 1** **j LOOP**

AFTER: # after code **DONE:** # done CODE

LAD 06

srli s2, s1, 1 } Count consec. 1s
and s3, s3, s2



Subroutine conventions

Arguments: use a0 to a7

Return in a0

Caller save: t0-t6, a0-a7

Callee save: s0-s11, ra, sp

CONFIG_KEYS

li t0, 0b1111 } sw t0, 8(t1) # interrupt

li t1, KEYS } sw t0, 12(t1) # reset edge

jr ra

Local variables
a0-a7, t0-t6
s0-s11
ra
additional arg

Recursion (N=N-1)

- start: li s9, N
lw a0, 0(s9)
jal FINDSUM

END: break **base case**

FINDSUM: bnez a0, RECURSE
jr ra

RECURSE: addi sp, sp, -8
sw a0, 4(sp)
sw ra, 0(sp)
addi a0, a0, -1
jal FINDSUM

EEF: lw t0, 4(sp) # Get N
N = N + Findsum(N-1)
add a0, a0, t0
lw ra, 0(sp)
add sp, sp, 8
jr ra

Bubblesort (Lab 07)

def bubblesort:

length = list[0]

for i in range(length):

for j in range(length-i-1):

SWAP(list[j])

def SWAP:

swapped = 0

if list[j] > list[j+1]

list[j], list[j+1] = list[j+1], list[j]

swapped = 1

return swapped

swap done sort-ing 3, 1, 3, 5, 8 ing

Polling (Timer)

TIMER

POLL: lw s6, 0(s0)

andi s6, s6, 1

bgez s6, POLL

...

sw zero, 0(s0)

j POLL

Polling (Key)

key

POLL: lw r2, 0(s0)

bgez r2, POLL

WAIT: lw s3, 0(s0)

bnez s3, WAIT

CHECK KEYS...

j POLL

CONFIG_TIMER

.equ PERIOD, number

.equ TIMER, 0xFFFF0000

config period

li t1, PERIOD

li t0, TIMER

li t3, 0xFFFF0000

srli t2, t1, 16

and t1, t1, t3

sw s1, 8(s0)

sw r2, 12(s0)

interrupt start

li t1, 0b111

sw t1, 4(s0)

jr ra

interrupt_handler

allocate stack ...

check

csrcr s0, mcause

li s1, 0x80000000 (timer)

[li s1, 0x80000000 (keys)]

bne s1, s0, return

CODE for interrupt...

Clear

li s0, DEVICE

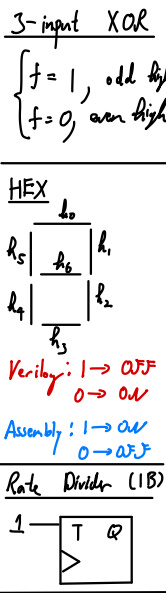
sw zero, 0(s0) (timer)

[li s1, 0b1111]

[sw s0, 12(s1)] (keys)

restore stack ...

mret



Interrupt

- start: li sp, 0x20000

① Turn off interrupts

csrcr mstatus, zero

② Config device

jal CONFIG_DEVICE

③ Enable interrupt, set address

li t0, 0x10000 # bit 16 for IRQ18 timer

or (li t0, 0x40000) for keys, IRQ18

{ csrcr mie, t0 # set

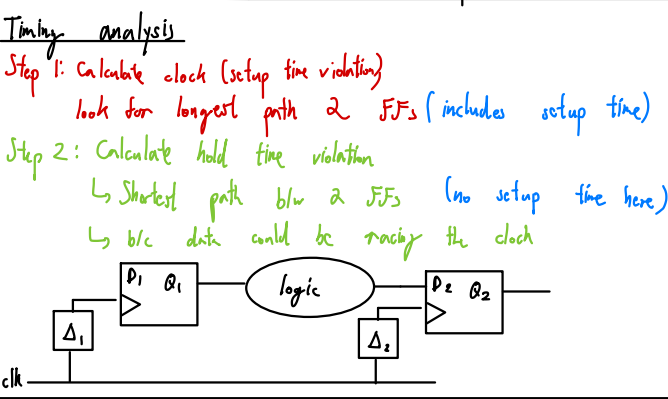
{ la t0, interrupt_handler_addr

{ csrcr mtimec, t0

{ li t0, 0b1000 # allow interrupt

{ csrcr mstatus, t0 again

END: j END # infinite loop



Setup time violation

↳ Logic too slow for correct value to arrive at input to FF on right (D2) before tsu, before clock edge

Constraint: $T_{min} \geq t_{ca} + t_{logic_max} + t_{su} + \Delta_1$ (for $\Delta_2 = 0, \Delta_1 \neq 0$)

Hold time Violation

↳ Ensure data input to FF remains stable for th after the positive edge of the clock

Constraint: $t_h + \Delta \leq t_{ca} + t_{logic_min}$ (for $\Delta_1 = 0, \Delta_2 \neq 0$)

Processor

OP CODE: II M X DDDD DDDD DDDD

II: instr.

X: dest. for reg. rX

M: Choose source reg/immediate value

{ M = 1 → D... = #D (immediate)

{ M = 0 → D... = 0000 0000 0001

Y = 0, rY = r0

Y = 1, rY = r1

Cycles (control signals)

C0 IRin = 1

C1 Set select based on X, Ain = 1

C2 Rin = 1, set select based on M, Y ALUOP based on II

C3 Select = 00, Rin on Rin based on X, Done = 1

FSM

If 1/0 occurs in w → z = 1, else z = 0

Step 1: State Diagram

Step 2: State Table

PS	w = 0	NS	w = 1	Output z
A	A	B	0	
B	C	B	0	
C	A	D	0	
D	C	B	1	

Step 3: State assignment

↳ Choose # of FFs (y2, y1)

↳ Choose state codes (A:00, B:01, C:10, D:11)

Step 4: State assigned Table (Follow Step 2)

PS	w = 0	NS	w = 1	Output z
y2 y1	y2 y1	y2 y1	y2 y1	z
0 0	0 0	0 1	0	
0 1	1 0	0 1	0	
1 0	0 0	1 1	0	
1 1	1 0	0 1	1	

Step 5: Synthesize circuit

a) Use boolean algebra to obtain: ns, output

b) Draw circuit

